

RAG Foundations

A Complete Knowledge Base for RAG Pipeline Evaluation

Covers all 10 evaluation topics · Retrieval · Chunking · RAGAS · Reranking · Hybrid Search

Contents

#	Topic	Chapter
01	Retrieval-Augmented Generation	1
02	RAG Pipeline Components	2
03	Bi-encoder vs Cross-encoder	3
04	BM25 and Hybrid Search	4
05	RAGAS Faithfulness	5
06	Context Precision	6
07	Context Recall	7
08	Semantic vs Fixed-size Chunking	8
09	Reciprocal Rank Fusion	9
10	LLM Temperature in Evaluation	10

Retrieval-Augmented Generation

What it is, why it exists, and how it works

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines a retrieval system with a large language model (LLM). Instead of relying solely on knowledge baked into the model's weights during training, a RAG system first searches an external knowledge base for documents that are relevant to the user's question, then passes those documents as additional context to the LLM before generating an answer.

The key insight is that LLMs have a fixed knowledge cutoff and can hallucinate facts. By grounding the generation step in retrieved documents, RAG dramatically reduces hallucination and allows the system to answer questions about information the model was never trained on.

Core definition: RAG = Retrieval + Augmented context + Generation. The retriever fetches relevant documents from an external knowledge base, and the LLM uses those documents as context to generate accurate, grounded answers.

Why RAG Exists

Traditional LLMs have three key limitations that RAG addresses:

- Knowledge cutoff: the model only knows what it saw during training. Any newer information is invisible to it.
- Hallucination: when a model does not know something, it may confidently generate plausible-sounding but incorrect information.
- Context window limits: even with a large context window, loading entire knowledge bases into every prompt is impractical and expensive.

RAG solves all three: the knowledge base is always up to date, answers are grounded in retrieved source documents (reducing hallucination), and only the relevant documents are loaded into context rather than everything at once.

How RAG Works — Step by Step

Step	What happens
1. User query	User submits a question or query.

2. Embed query	The query is converted into a dense vector embedding using an embedding model.
3. Vector search	The query embedding is compared against stored document embeddings in a vector database.
4. Augment prompt	The retrieved chunks are inserted into the LLM's prompt as context alongside the original query.
5. Generate answer	The LLM generates an answer using both its parametric knowledge and the retrieved context.
6. Return response	The answer (and optionally source citations) is returned to the user.

RAG vs Pure LLM

Property	Pure LLM	RAG System
Knowledge freshness	Fixed at training cutoff	Always current (live knowledge base)
Hallucination risk	High on unknown facts	Lower — grounded in retrieved docs
Transparency	Black box	Can cite source documents
Scalability	Requires retraining to update	Update knowledge base only
Cost per query	Low	Slightly higher (retrieval + LLM)

RAG Pipeline Components

The five building blocks every RAG system is made of

Overview

A RAG pipeline consists of five main components: a document loader, a text splitter for chunking, an embedding model, a vector database for storage and retrieval, and a language model for answer generation. Understanding each component helps you diagnose and improve your system's performance.

1. Document Loader	Ingests raw source material — PDFs, web pages, databases, markdown files, etc. — and converts it into plain text Documents that downstream components can process. Examples: PyPDFLoader, WebBaseLoader, CSVLoader in LangChain.
2. Text Splitter (Chunker)	Breaks long documents into smaller chunks that fit within the embedding model's context window and contain coherent units of information. The chunking strategy has a major impact on retrieval quality. Covered in depth in Chapter 8.
3. Embedding Model	Converts text chunks into dense vector representations (embeddings). Semantically similar texts produce similar vectors. Common choices: OpenAI text-embedding-3-small, sentence-transformers/all-MiniLM-L6-v2. The embedding model used at indexing time must match the one used at query time.
4. Vector Database	Stores chunk embeddings and enables fast approximate nearest-neighbour search. When a query embedding arrives, the vector DB finds the most similar stored embeddings and returns those chunks. Examples: ChromaDB, FAISS, Pinecone, Weaviate.

5. Language Model (Generator)	Takes the original query plus the retrieved context chunks and generates a final natural language answer. The LLM is responsible for synthesis — combining retrieved facts into a coherent, readable response. Examples: GPT-4o, Claude 3, Llama 3.
--------------------------------------	---

How the Components Connect

At indexing time, documents flow through: Loader → Splitter → Embedding Model → Vector DB. At query time, the flow is: Query → Embedding Model → Vector DB (search) → top-K chunks → LLM (generate) → Answer. The embedding model is the bridge between both phases, which is why using different models at index and query time breaks the system.

Bi-encoder vs Cross-encoder

Two approaches to relevance scoring — and why you need both

Bi-encoder (Dense Retrieval)

A bi-encoder encodes the query and each document independently into separate fixed-size vector embeddings using the same encoder model. Relevance is then estimated by computing the dot product or cosine similarity between the query vector and each document vector.

Because the query and documents are encoded separately, you can pre-compute and store all document embeddings at indexing time. At query time, only the query needs to be embedded — making retrieval extremely fast even over millions of documents.

Bi-encoder speed: similarity between query and document is a single dot product — $O(1)$ per document after embeddings are stored. This is why bi-encoders power vector databases like ChromaDB and FAISS.

Limitation: because the query and document never attend to each other's tokens, the model can miss fine-grained interactions. For example, the word 'not' in a query can completely change its meaning, but a bi-encoder may still return a high-similarity document that ignores the negation.

Cross-encoder (Re-ranking)

A cross-encoder processes the query and a candidate document together as a single concatenated input. Every token in the query can attend to every token in the document via the full self-attention mechanism. This produces a much more accurate relevance score because the model sees all token-level interactions between query and document.

The tradeoff is speed: because you cannot pre-compute scores, every (query, document) pair must be scored at query time. Scoring 1 million documents for every query is infeasible, so cross-encoders are used as re-rankers — they re-score a small candidate set (typically top 20 from a bi-encoder) rather than the full corpus.

The Two-Stage Retrieval Pattern

Method	Purpose
Bi-encoder + vector DB	Fast broad retrieval — fetch top 20-100 candidates from corpus
Cross-encoder re-ranker	Accurate re-scoring of candidates — pick top 3-5 to send to LLM

The recommended cross-encoder model for re-ranking is cross-encoder/ms-marco-MiniLM-L-6-v2, trained on the MS MARCO passage ranking dataset. It is fast (scores ~50 passages in ~50ms), free, runs locally, and provides strong re-ranking performance. Upgrading to the L-12-v2 variant adds 2-3 percentage points of ranking accuracy at roughly double the compute cost.

Property	Bi-encoder	Cross-encoder
Input	Query encoded alone; doc encoded alone	Query + doc concatenated together
Speed	Very fast — pre-compute doc embeddings	Slow — must score each pair at runtime
Accuracy	Good for broad recall	High — captures token interactions
Use case	First-stage retrieval from large corpus	Re-ranking a small candidate set
Storage	Stores all doc embeddings	No pre-computation possible

BM25 and Hybrid Search

Combining keyword and semantic retrieval for maximum coverage

What is BM25?

BM25, short for Best Match 25, is a probabilistic keyword-based ranking algorithm that scores documents based on three factors: term frequency, inverse document frequency, and document length normalization. It is the successor to TF-IDF and has been the standard retrieval algorithm in search engines for decades.

The BM25 Formula

$$\text{Score}(D, Q) = \text{sum over terms } t \text{ in } Q \text{ of: } \text{IDF}(t) * \left[\frac{\text{TF}(t, D) * (k_1 + 1)}{\text{TF}(t, D) + k_1 * (1 - b + b * |D| / \text{avgdl})} \right]$$

Where:

- **TF(t,D):** how many times term *t* appears in document *D*
- **IDF(t):** $\log \frac{(N - df + 0.5)}{(df + 0.5)}$ — how rare term *t* is across all *N* documents (*df* = document frequency)
- **k₁:** term frequency saturation parameter, typically 1.2-2.0. Controls how much repeated terms boost the score
- **b:** length normalisation parameter, typically 0.75. *b*=1 fully normalises by length, *b*=0 disables it
- **|D|/avgdl:** ratio of document length to average document length

Why BM25 is Used in Hybrid Search

Semantic vector search is excellent at understanding meaning and intent, but it can miss exact keyword matches. Consider a query like 'ISO 27001 clause 6.1.2' — the numbers and specific identifiers matter exactly. A semantic search might retrieve documents about information security in general, while BM25 will directly find documents containing that exact clause identifier.

Conversely, BM25 fails when synonyms or paraphrasing are involved. A query for 'car' will not match a document that only uses 'automobile'. Semantic search handles this naturally because both words map to nearby vectors.

Rule of thumb: if the query contains specific identifiers, codes, names, or acronyms, BM25 wins. If the query is a natural language question about a concept, semantic search wins. Hybrid uses both, so you never have to choose.

Hybrid Search with Reciprocal Rank Fusion

Hybrid search runs BM25 and vector search in parallel and fuses the ranked result lists using Reciprocal Rank Fusion (RRF). Each retrieved document gets a combined score based on its rank position in both lists. Documents that appear highly ranked in both lists receive the highest final scores.

This is more powerful than simply averaging raw scores, because BM25 and vector similarity scores live on different scales and cannot be directly compared. RRF only uses rank positions, which are naturally comparable. Chapter 9 covers RRF in full detail.

RAGAS — Faithfulness

Measuring whether the answer is grounded in retrieved context

What is Faithfulness?

Faithfulness is one of the five RAGAS evaluation metrics. It measures whether all claims made in the generated answer can actually be inferred from the provided context documents. A faithful answer contains only statements that are supported by the retrieved context — it does not introduce external knowledge, invented facts, or hallucinations.

Faithfulness score = (number of claims in answer that are supported by context) / (total number of claims in the answer). Score range: 0 to 1, higher is better.

How RAGAS Computes Faithfulness

RAGAS measures faithfulness through a multi-step LLM-as-judge process:

- Step 1 — Decompose: the generated answer is broken into individual atomic statements. For example, 'Paris is the capital of France and has a population of 2 million' becomes two separate claims.
- Step 2 — Verify: each claim is checked against the retrieved context using an LLM judge. The judge determines whether the claim can be directly inferred from the context (true) or not (false).
- Step 3 — Score: faithfulness = supported claims / total claims.

What Causes Low Faithfulness?

- The LLM is adding information from its training data that was not in the retrieved documents.
- The retriever is returning irrelevant chunks that do not support the answer, so the LLM fills in gaps with hallucinations.
- The chunk size is too small — retrieved chunks are missing context needed to support the full answer.
- The LLM temperature is too high, causing creative generation that drifts from the source material.

Fix low faithfulness: set LLM temperature=0, increase retrieved context size (top_k), improve chunk quality, and add an instruction in the system prompt: 'Answer only based on the provided context. Do not add information from your training data.'

RAGAS — Context Precision

Measuring retrieval signal-to-noise ratio

What is Context Precision?

Context precision measures the proportion of retrieved context chunks that are actually relevant to answering the question. It is a measure of retrieval precision — the signal-to-noise ratio of what the retriever returned.

A high context precision means the retriever returned mostly useful, on-topic chunks with little noise. A low score means many retrieved chunks were irrelevant, forcing the LLM to sift through a lot of useless information — which can confuse it and degrade answer quality.

Context Precision = (number of relevant chunks retrieved) / (total chunks retrieved).
RAGAS uses a weighted variant that also considers the ranking — relevant chunks ranked higher contribute more to the score.

Context Precision vs Context Recall

Precision and recall represent a classic trade-off. A retriever that returns only one highly relevant chunk will have perfect precision but potentially low recall (if other relevant chunks exist but were not retrieved). A retriever that returns 50 chunks guarantees high recall but likely low precision (most chunks will be irrelevant).

In practice, increasing top_k (number of retrieved chunks) improves recall but hurts precision. Re-ranking with a cross-encoder helps restore precision by filtering out irrelevant candidates after a broad retrieval.

Improving Context Precision

- Use a cross-encoder re-ranker to filter out low-relevance chunks after initial retrieval.
- Tune the similarity threshold — only return chunks above a minimum similarity score.
- Improve your chunking strategy so chunks are semantically coherent (see Chapter 8).
- Use metadata filtering — if you know the document type or date, pre-filter before vector search.

RAGAS — Context Recall

Measuring whether the retriever found everything it needed

What is Context Recall?

Context recall measures how much of the information needed to produce the ground truth answer is present in the retrieved context. If the correct answer requires five key facts and the retrieved chunks contain only three of them, context recall is $3/5 = 0.60$.

A high context recall means the retriever successfully captured all the relevant information from the knowledge base. A low context recall means the system is likely to give incomplete or incorrect answers because it simply did not retrieve the supporting information.

Context Recall = (number of ground truth statements that can be attributed to the retrieved context) / (total number of statements in the ground truth).

What Causes Low Context Recall?

- The relevant document was never ingested into the knowledge base in the first place.
- The chunk that contains the answer exists, but was not retrieved because it was not sufficiently similar to the query.
- The chunk size is too small, splitting the answer across multiple chunks that are individually not similar enough to the query.
- The embedding model does not capture the semantic relationship between the query and the relevant document.
- `top_k` is set too low — the correct chunk exists but is ranked 5th and only top 4 are retrieved.

Improving Context Recall

- Increase `top_k` to cast a wider retrieval net (at the cost of precision — use re-ranking to compensate).
- Use larger chunk overlap so that information spanning chunk boundaries is not lost.
- Use hybrid search (BM25 + vector) to improve coverage for keyword-specific queries.
- Audit your document ingestion pipeline — ensure all relevant source documents are indexed.

Chunking Strategies

How you split documents determines what gets retrieved

Why Chunking Matters

Chunking is the process of splitting source documents into smaller pieces before embedding and storing them. The chunking strategy is one of the most impactful decisions in a RAG pipeline: it determines the granularity of retrieval, the coherence of individual chunks, and how well the retrieved context supports the LLM's generation.

Strategy 1: Fixed-size Chunking

Fixed-size chunking splits text at fixed token intervals regardless of meaning. It is the simplest and fastest strategy — a sliding window of N tokens with optional overlap between windows.

- Advantage: extremely fast, simple to implement, no external dependencies.
- Disadvantage: can cut mid-sentence, mid-paragraph, or even mid-word. The resulting chunks may contain incomplete thoughts that are hard to match against queries.
- Best for: baseline comparison, very uniform text like structured data exports.

Typical settings: `chunk_size=512` tokens, `chunk_overlap=0` or `64` tokens.

Strategy 2: Recursive Character Splitting

Recursive chunking tries to split on natural text boundaries in order of preference: paragraph breaks (`\n\n`), then line breaks (`\n`), then sentence endings (`. ? !`), then spaces, then individual characters as a last resort. It only falls back to the next separator when the current chunk would exceed `chunk_size`.

- Advantage: much better at preserving coherent units of meaning compared to fixed-size. Respects the document's natural structure.
- Disadvantage: still does not understand meaning — splits are based on formatting, not semantics.
- Best for: general purpose use. This is the recommended default for most RAG applications.

Typical settings: `chunk_size=512`, `chunk_overlap=64`. The overlap ensures context is not lost at boundaries.

Strategy 3: Semantic Chunking

Semantic chunking groups sentences based on embedding similarity. Each sentence is embedded individually, and a chunk boundary is inserted only when the cosine similarity between adjacent sentences drops below a threshold — indicating a shift in topic or meaning.

- Advantage: each chunk contains a coherent unit of meaning. The semantic shift detection ensures you never cut in the middle of an idea.
- Disadvantage: requires embedding every sentence at indexing time, making it slower and more expensive than the other strategies. Chunk sizes vary and can be unpredictable.
- Best for: technical documents, research papers, reports where meaning shifts are meaningful boundaries.

Strategy Comparison

Property	Fixed-size	Recursive	Semantic
Speed	Fastest	Fast	Slow (embeds each sentence)
Coherence	Low	Medium	High
Context Precision	Lower	Medium	Higher
Chunk count	Predictable	Roughly predictable	Unpredictable
Recommended for	Baselines	General use	Technical docs

Reciprocal Rank Fusion

Combining multiple ranked lists into one superior ranking

What is Reciprocal Rank Fusion?

Reciprocal Rank Fusion (RRF) is a technique for combining rankings from multiple retrieval systems into a single unified ranking. It was introduced by Cormack, Clarke, and Buettcher in 2009 and remains the standard method for hybrid search fusion because of its simplicity and consistently strong empirical performance.

The RRF Formula

$$\text{RRF_score}(d) = \text{sum over all ranked lists } L: 1 / (k + \text{rank_L}(d))$$

Where:

- **d**: a document being scored
- **rank_L(d)**: the position of document d in ranked list L (1 = top result)
- **k**: a smoothing constant, typically set to 60. It prevents the top-ranked results from dominating too strongly

How RRF Works — Worked Example

Document	Vector rank	BM25 rank	RRF score (k=60)
Doc A	1	3	$1/(60+1) + 1/(60+3) = 0.01639 + 0.01587 = 0.03226$
Doc B	2	1	$1/(60+2) + 1/(60+1) = 0.01613 + 0.01639 = 0.03252$
Doc C	3	10	$1/(60+3) + 1/(60+10) = 0.01587 + 0.01429 = 0.03016$
Doc D	50	2	$1/(60+50) + 1/(60+2) = 0.00909 + 0.01613 = 0.02522$

Doc B wins even though it was ranked 2nd in vector search, because it was top in BM25. Doc D was ranked 50th in vector search but 2nd in BM25 — its strong keyword relevance saves it. This is exactly the benefit of hybrid search: good results from either system get surfaced in the final ranking.

Why k=60?

The constant k=60 was found empirically by the original RRF authors to perform well across a wide range of retrieval tasks. A smaller k gives more weight to top-ranked results (making the

fusion more aggressive). A larger k flattens the differences between ranks. $k=60$ is almost universally used as the default.

RRF vs Score Normalisation

An alternative to RRF is to normalise raw scores (e.g., cosine similarities and BM25 scores) to the same scale and then average them. This is simpler but has a key weakness: BM25 scores and cosine similarities have different distributions that are hard to make comparable. RRF sidesteps this entirely by only using rank positions, which are naturally comparable across any retrieval system.

LLM Temperature in RAG Evaluation

Why determinism is non-negotiable for reproducible evaluation

What is Temperature?

Temperature is a parameter that controls the randomness of an LLM's output. It is applied to the logits (raw output scores) of the model before sampling the next token. A temperature of 1.0 leaves the distribution unchanged. A temperature below 1.0 makes the distribution sharper — the most probable token becomes even more probable. A temperature above 1.0 flattens the distribution, increasing randomness and creativity.

Temperature = 0 Means Deterministic Output

When temperature is set to 0, the model always selects the highest-probability token at each step, producing greedy decoding. Given the same input, the model will always produce exactly the same output. This is also called greedy decoding.

Critical rule: always set temperature=0 when evaluating a RAG pipeline. Differences in RAGAS scores across runs must reflect changes in the retrieval pipeline, not random variation in generation.

Why Temperature=0 is Required for RAG Evaluation

Imagine you run your RAG pipeline on the same 10 questions twice and get different RAGAS scores. Is that because you improved the chunking strategy? Or because the LLM happened to generate a slightly different answer the second time due to randomness? You cannot tell unless generation is deterministic.

With temperature=0, any change in RAGAS scores across two runs is guaranteed to be caused by a change in the pipeline (chunking, retrieval, re-ranking) rather than generation variance. This makes your evaluation rigorous and reproducible.

- Reproducibility: same query + same context always produces same answer.
- Attributability: score changes are caused by pipeline changes, not generation noise.
- Comparability: you can fairly compare run A vs run B knowing generation is held constant.
- Debugging: when a score drops, you know exactly where to look — the retrieval side.

When to Use Non-Zero Temperature

Temperature above 0 is appropriate for production use where answer diversity and naturalness matter. Typical production values are 0.1-0.3. Values above 0.7 are suited for creative tasks. However, for any systematic evaluation or A/B testing of pipeline variants, temperature must always be 0.

Temperature	Behaviour	Use case
0.0	Fully deterministic, greedy	Evaluation, testing, A/B comparison
0.1-0.3	Near-deterministic, small variation	Production RAG — factual Q&A
0.5-0.7	Moderate creativity	Production — conversational assistants
0.8-1.0	High variation, creative	Brainstorming, creative writing
>1.0	Very random, often incoherent	Rarely used in practice

Quick Reference

All 10 evaluation questions with their ground truth answers — directly from your `test_dataset.json`.

Q1: What is retrieval-augmented generation?

Retrieval-augmented generation (RAG) is a technique that combines a retrieval system with a large language model. The retriever fetches relevant documents from an external knowledge base, and the LLM uses those documents as context to generate accurate, grounded answers.

Q2: What are the main components of a RAG pipeline?

A RAG pipeline consists of five main components: a document loader, a text splitter for chunking, an embedding model, a vector database for storage and retrieval, and a language model for answer generation.

Q3: What is the difference between a bi-encoder and a cross-encoder?

A bi-encoder encodes the query and document independently into separate embeddings and compares them via dot product or cosine similarity. A cross-encoder processes the query and document together as a single input, allowing the model to capture fine-grained token-level interactions, making it more accurate but slower.

Q4: What is BM25 and why is it used in hybrid search?

BM25 (Best Match 25) is a probabilistic keyword-based ranking algorithm that scores documents based on term frequency, inverse document frequency, and document length normalization. It is used in hybrid search because it excels at exact keyword matches that semantic vector search may miss.

Q5: How does RAGAS measure faithfulness?

RAGAS measures faithfulness by checking whether each claim in the generated answer can be inferred from the provided context documents. It decomposes the answer into individual statements and verifies each statement against the retrieved context using an LLM judge.

Q6: What is context precision in RAGAS?

Context precision measures the proportion of retrieved context chunks that are actually relevant to answering the question. A high context precision means the retriever returned mostly useful chunks with little noise.

Q7: What is context recall in RAGAS?

Context recall measures how much of the information needed to produce the ground truth answer is present in the retrieved context. A high context recall means the retriever captured all the relevant information from the knowledge base.

Q8: What is semantic chunking and how does it differ from fixed-size chunking?

Semantic chunking groups sentences based on embedding similarity, inserting chunk boundaries only when the meaning shifts significantly. Fixed-size chunking splits text at fixed token intervals regardless of meaning, which can cut mid-sentence and disrupt coherent ideas.

Q9: What is Reciprocal Rank Fusion?

Reciprocal Rank Fusion (RRF) is a technique for combining rankings from multiple retrieval systems. It assigns each document a score of $1/(k + \text{rank})$ where k is a smoothing constant (typically 60) and rank is the document's position in each list. Documents that rank highly in multiple lists receive the highest combined scores.

Q10: Why should the LLM temperature be set to 0 during RAG evaluation?

Setting temperature to 0 makes the LLM deterministic, producing the same answer for the same input every time. This is critical for evaluation because it ensures that differences in RAGAS scores across runs reflect changes in the retrieval pipeline rather than random variation in generation.

This document covers every topic tested in `eval/test_dataset.json`. Ingest it as your sample PDF in `eval/sample_docs/` and run: `python eval/run_eval.py --pdf eval/sample_docs/RAG_Foundations.pdf --strategy recursive --retriever rerank`